IRWA Final Project 2026 – Part 2: Indexing and Evaluation

Antonio Javier Ortiz Gil (u233542) Juan Antonio Oliver Blanes (u232701) Aitor Bolige Cubas (u232752)
GitHub: `https://github.com/ToniOrtizGil/irwa-search-engine-2026` Tag: `IRWA-2026-part-2`

Code. `myapp/search/algorithms.py` (inverted index, AND search and TF-IDF ranking), `myapp/evaluation/metrics.py` (evaluation metrics) and the notebook `part2_indexing_evaluation.ipynb`, which generates all the numbers of this report. The notebook, our relevance judgments (`own_queries_labels.csv`) and a README that explains how to run it are in `project_progress/part_2/`.

1 Part 1: Indexing

1.1 Inverted index

We build the inverted index as in the lab: a dictionary that maps each term to the list of documents where it appears, together with its positions. We index the combined field `search_tokens` of Part 1 (title + description + brand + sub_category + product_details values), because it contains the words that users write in fashion queries (for example, the material only appears in `product_details`). The terms are the ones produced by `build_terms()` of Part 1, so the index only contains stemmed terms (there is an entry for *jean*, but not for *jeans*). While building the index we also compute, as in the theory of the course (log base 2), the document frequency df_t, $idf_t = \log_2(N/df_t)$, the tf-idf weight of each term in each document, $w_{t,d} = (1 + \log_2 f_{t,d}) \cdot idf_t$ if $f_{t,d} > 0$ (0 otherwise), where $f_{t,d}$ is the number of positions of the term in the document, and the length of each document $|d| = \sqrt{\sum_t w_{t,d}^2}$ (over all its terms). With $N = 28{,}080$ documents, the index has 5,733 terms and it is built in about 4 seconds.

1.2 Conjunctive queries

The query is pre-processed with the same `build_terms()` function as the documents. Since the queries are conjunctive (AND), a document is returned only if it contains **all** the query terms, so we take the **intersection** of the posting lists of the terms (in the lab it was the union, OR). If one of the terms is not in the index, the query returns no documents. For example, *women full sleeve sweatshirt cotton* returns 484 documents and *men slim jeans blue* returns 221.

1.3 Test queries

To choose our five queries we looked at the most frequent terms of the titles (document frequency): *women* (13,041 titles), *men* (13,021), *tshirt*, *neck*, *solid*, *round*, *print*, *blue*, *black*... We combined easy queries made of these popular terms with harder ones that we expected to fail, so that the evaluation can show the problems of the system (Table 1).

Id	Query	Results	Why we chose it
3	women black round neck tshirt	1,112	Only very popular terms; the most common type of product in the corpus.
4	men checkered casual cotton shirt	346	Five attributes; the material (*cotton*) is mainly in `product_details`.
5	women printed kurta	42	Ethnic product with a rare term (*kurta*, df = 1,196).
6	men white sneakers	66	Footwear, a different category (only 920 products); rare term *sneaker*.
7	women v neck tshirt	6,351	Expected to fail: the "v" is a one-character token removed by our pre-processing.

Table 1: Our five test queries (ids 3–7; ids 1 and 2 are the validation queries).

1.4 TF-IDF ranking

We use the vector model of the theory: the query is weighted with the same formula as the documents, $w_{t,q} = (1 + \log_2 f_{t,q}) \cdot idf_t$, and each returned document gets the cosine similarity $cos(q, d) = \sum_{t \in q} w_{t,q}\, w_{t,d} \,/\, (|q|\,|d|)$ (only the query terms contribute to the product; $|q|$ is the same for all the documents, so it does not change the order). We checked the implementation with the two examples solved in class: for *to do* we get d1 0.660 > d2 0.408 > d3 0.118 > d4 0.058 (without dividing by $|q|$) and for *sol playa* the cosines 0.982, 0.735, 0.147 and 0. Rare terms (high idf, like *kurta* or *sneaker*) weigh more than common ones (like *women*). Many products have exactly the same title and attributes, so they get the same score; in that case we sort them by `pid`, so the ranking is always the same.

2 Part 2: Evaluation

2.1 Metrics

The metrics are implemented in `myapp/evaluation/metrics.py`, following the Evaluation lab and the slides. They receive the relevance labels (1/0) of the returned documents in ranking order. We use $K = 10$:

- **P@K** = relevant documents in the top K / K.
- **R@K** = relevant documents in the top K / total relevant documents of the query.
- **AP@K** = average of P@i at the positions $i \leq K$ of the relevant documents. As in the slides (MAP summary, "unretrieved relevant documents"), a relevant document that is not in the top K counts with precision 0, so the

sum is divided by the **total** number of relevant documents of the query.

- **F1@K** $= 2 \cdot P@K \cdot R@K/(P@K + R@K)$.
- **MAP** = mean of AP@K over the queries; **MRR** = mean of $1/rank$ of the first relevant document in the top K (0 if there is none).
- **NDCG@K** = DCG@K / IDCG@K, with $DCG@K = \sum_{i=1}^{K} rel_i / \log_2(i+1)$ (formula of the slides) and IDCG@K the DCG of the ideal ranking (all the relevant documents first). Our labels are binary, so $rel_i \in \{0, 1\}$.

We checked AP and MAP with the examples of the slides: we get 0.775 and 0.521 for the two rankings of the Average Precision example (slide: 0.78 and 0.52) and 0.533 for the MAP example (slide: 0.53).

2.2 Results for the validation queries

Query	P@10	R@10	AP@10	F1@10	NDCG@10
1: women full sleeve sweatshirt cotton	1.000	0.769	0.769	0.870	1.000
2: men slim jeans blue	0.600	0.600	0.569	0.600	0.716
MAP@10	0.669				
MRR@10	1.000				

Table 2: Evaluation with `validation_labels.csv` ($K = 10$).

2.3 Our relevance judgments

We acted as expert judges for our five queries: we labelled the **top 20** documents returned by our system for each query with 1 (relevant) or 0 (non-relevant), with a strict criterion: a document is relevant only if it matches everything the query asks for (type of product, gender, color, pattern, neck type, material…). For example, for *women printed kurta* the "Cotton Printed Kurta Fabric (Unstitched)", the five "Women Kurta and Pyjama Set" products (a set, not a kurta) and a "Self Design" (not printed) kurta are not relevant, and for *men white sneakers* the white t-shirts and the "Sneakers For Women (White)" are not relevant. We reviewed the doubtful labels with the product data: in query 4 one shirt is "Pure Cotton" and the other 19 are "Cotton Blend", and we count all of them as relevant because they contain cotton; in query 5 the five sets have `Pattern` = "Printed", but they stay non-relevant because they are a set, and the "Self Design" kurta stays non-relevant although its description says "Printed" (the data is not always consistent). The number of relevant documents in the top 20 is 20, 20, 13, 1 and 1 for queries 3 to 7. Since we only judged the top 20, the recall is computed with respect to the relevant documents found in these 20.

Query	P@10	R@10	AP@10	F1@10	NDCG@10
3: women black round neck tshirt	1.000	0.500	0.500	0.667	1.000
4: men checkered casual cotton shirt	1.000	0.500	0.500	0.667	1.000
5: women printed kurta	0.600	0.462	0.403	0.522	0.696
6: men white sneakers	0.000	0.000	0.000	0.000	0.000
7: women v neck tshirt	0.100	1.000	0.250	0.182	0.431
MAP@10	0.331				
MRR@10	0.650				

Table 3: Evaluation of our test queries ($K = 10$, top 20 judged).

2.4 Comments on the metrics and analysis of the results

How the validation queries are evaluated. Only 20 documents are judged for each query, so, as in the Evaluation lab, we rank these 20 documents with their tf-idf score. A judged document that does not contain all the query terms is not returned by the AND search, so it is not part of the ranking, but if it is relevant it still counts in the total number of relevant documents. For query 1 the 20 judged documents are returned; for query 2 only 8 are returned (6 of the 10 relevant ones, with non-relevant documents at ranks 5 and 8).

- **P@K** tells how many of the first K results are useful for the user, but it ignores their order inside the top K and how many relevant documents exist. Queries 1, 3 and 4 have P@10 = 1: the first page is perfect. Query 5 (0.6) mixes kurtas with fabrics and sets, and queries 6 (0) and 7 (0.1) show a first page that is almost useless.
- **R@K** tells how much of the relevant set we found. If a query has more relevant documents than K, it cannot reach 1: queries 3 and 4 have 20 relevant documents in the top 20, so R@10 = 0.5 even with a perfect top 10. Query 7 has R@10 = 1 with only P@10 = 0.1, because there is only 1 relevant document in its top 20. For query 2, R@10 = 0.6 because 4 relevant documents are not returned at all (Section 2.5).
- **F1@K** combines both, so it is only high when precision and recall are high at the same time (best: query 1, 0.870; query 7 gets only 0.182 although its R@10 is 1).
- **AP@K** also looks at the order: relevant documents at the top give more. With the definition of the slides it is divided by the total number of relevant documents, so it is limited by the recall: in queries 3 and 4, AP@10 = R@10 = 0.5. In query 2, AP@10 (0.569) is lower than R@10 (0.6) because a non-relevant document at rank 5 comes before two relevant ones. In query 7 the only relevant document is at rank 4, so AP@10 = 1/4.

- **MAP** summarizes AP for a set of queries with one number: 0.669 for the validation queries and 0.331 for ours, pulled down by queries 6 and 7. It is useful to compare systems, but it hides which queries fail.
- **MRR** only looks at the first relevant document, which is what matters when the user only needs one good result. It is 1 for the validation queries and 0.65 for ours: in query 6 the first relevant document is at rank 12 (outside the top 10, so it adds 0), in query 7 it is at rank 4 (it adds 1/4) and in the other queries it is at rank 1.
- **NDCG@K** discounts the gain of each relevant document by its position and normalizes by the ideal ranking inside the top K. Unlike AP@K, a perfect top 10 gives 1 even if more relevant documents exist (queries 3 and 4), so it measures the quality of the order rather than the coverage. For this reason it can be very different from AP: in queries 3 and 4, NDCG@10 = 1 but AP@10 = 0.5. With graded labels it would also reward putting the most relevant documents first.

Overall, the system works well when the query uses the same words as the titles (queries 1, 3 and 4), and it fails when a word is lost in the pre-processing, when a query word appears in a field that is not about the product, when the product data is noisy or when the AND is too strict.

2.5 Problems of the current system and proposed solutions

1. **The AND is too strict** (retrieval accuracy, query formulation). In query 2, 4 relevant judged documents are not returned: 3 "Women Blue Jeans" (judged relevant but without *men*) and a "Tapered Fit Men Blue Jeans" (without *slim*). *Solution*: rank documents that contain most of the terms (OR with ranking, or relax the AND when there are few results).
2. **All fields have the same weight** (handling of fields, ranking quality). A word in the description counts like a word in the title. In query 6 only 5 of the 66 results are sneakers (ranks 9, 12, 27, 38 and 56): the first results are white t-shirts whose description says "pair it with ... sneakers". Moreover, the first sneakers are "Sneakers For Women (White)", which match *men* only because their `sub_category` is "Men's Footwear" (182 of the 920 products of this sub_category are for women). *Solution*: index the fields separately (we already store them since Part 1), give more weight to the title and `product_details`, and use the attribute "Ideal For" for the gender.
3. **Short documents get higher scores** (ranking quality). The cosine divides by the length $|d|$, so a short document (usually without description) beats a longer one that contains the same query terms: in query 3, the top 10 has 23.8 terms on average against 51.6 for all the results, none of them has a description (31% of all the results have none); queries 4 and 7 show the same. Products with a complete description are pushed down even if they are relevant. *Solution*: a ranking with a softer length normalization and tf saturation, like BM25 (Part 3).
4. **Words lost in the pre-processing** (indexing strategy). The "v" of "v neck" is removed as a one-character token, so query 7 is really *women neck tshirt* and returns round and polo neck t-shirts. *Solution*: keep known compound terms ("v neck" → *vneck*, like we did with *tshirt*) or index bigrams.
5. **Cut brand names** (handling of fields). The `brand` field contains "REEB", and only 159 of the 802 REEB products contain the word *reebok* in another field, so *reebok women track suit* returns 0 results although there are 4 REEB women track suits. *Solution*: a dictionary of brand names and synonyms (also for words like *hoodie*/*sweatshirt* or *tee*/*tshirt*) for query expansion.
6. **Ties** (ranking quality). Many products have the same title and attributes, so they get the same score: in query 4 the 20 first results only have 5 different scores, so the order inside each group is arbitrary (by pid). *Solution*: break ties with other signals, like the rating, the discount or hiding the products out of stock (numeric fields of Part 1).
7. **Numeric fields are not used** (handling of fields). The user cannot filter by price, rating or stock. *Solution*: filters and a ranking that combines the text score with these fields (Part 3).
8. **Evaluation limits.** Our labels only cover the top 20 of each query, so our recall is relative (for example, there are more relevant printed kurtas after rank 20), and the labels depend on our criterion. *Solution*: judge more documents (for example, the results of several systems together) and use more judges.

Use of AI tools

We used Claude (Anthropic) to help write and debug the code, to pre-label the top 20 documents of our test queries (which we reviewed), and to draft parts of this report. All the code was reviewed and run by the group, all numbers come from our own execution of the notebook, and the decisions and explanations were checked and edited by the group members, who take full responsibility for the content.